Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Information and Communication Technology

# UNIT-4
# BASICS OF R:

## Study Guide

**Dr. Nirav Mehta**
**Assistant Professor**
**CSE, PIT**
**Parul University**

INDEX

**BASICS OF R**

**Introduction to R**

R is an open-source programming language mainly used for statistical computing, data analysis, machine learning, and visualization. It is widely used in Data Science because of its powerful libraries and analytical capabilities.

R provides:

- Statistical analysis
- Data visualization
- Machine learning algorithms
- Data manipulation
- Predictive modeling

**R Basics**

R programming is simple and interactive. Commands are executed directly in the R console or through RStudio.

**Simple R Program**

```
print("Welcome to R Programming")
```

**Output**

```
[1] "Welcome to R Programming"
```

**Data Types in R**

Data types specify the kind of value stored in a variable.

| Data Type | Description | Example |
|---|---|---|
| Numeric | Decimal numbers | 10.5 |
| Integer | Whole numbers | 5L |
| Character | Text/String | "R Language" |
| Logical | TRUE/FALSE values | TRUE |
| Complex | Complex numbers | 2+3i |

**Example of Data Types**

```
x <- 10.5

y <- 5L

z <- "Data Science"

a <- TRUE

b <- 3 + 2i

print(x)

print(y)

print(z)

print(a)

print(b)
```

**Objects in R**

Objects are variables used to store data.

**Creating Objects**

```
name <- "Nirav"

age <- 35

print(name)

print(age)
```

**Types of Objects in R**

| Object Type | Description |
|---|---|
| Vector | Stores same type elements |
| Matrix | Two-dimensional data |
| List | Stores multiple data types |
| Array | Multi-dimensional data |
| Data Frame | Tabular data |

**Vector in R**

A vector stores multiple values of the same type.

**Example**

```
marks <- c(70, 80, 90, 85)

print(marks)
```

**Matrix in R**

Matrix contains rows and columns.

**Example**

```
mat <- matrix(c(1,2,3,4,5,6), nrow=2, ncol=3)

print(mat)
```

**List in R**

Lists can store different data types together.

**Example**

```
student <- list(
  name = "Rahul",
  age = 21,
  marks = 85
)

print(student)
```

**Control Structures in R**

Control structures control the execution flow of a program.

**if Statement**

```
x <- 10

if(x > 5){

  print("x is greater than 5")

}
```

**if-else Statement**

```
marks <- 40

if(marks >= 50){

  print("Pass")

} else {

  print("Fail")

}
```

**else-if Ladder**

```
marks <- 75


if(marks >= 90){

  print("Grade A")

} else if(marks >= 70){

  print("Grade B")

} else {

  print("Grade C")

}
```

**for Loop**

```
for(i in 1:5){
```

```
 print(i)
}
```

**while Loop**

```
x <- 1

while(x <= 5){
 print(x)
 x <- x + 1
}
```

**repeat Loop**

```
x <- 1
repeat{
 print(x)
 x <- x + 1

 if(x > 5){
  break
 }
}
```

**break and next**

**break Example**

```
for(i in 1:10){
 if(i == 5){
  break
 }
 print(i)
```

```
}
```

**next Example**

```
for(i in 1:5){
  if(i == 3){
    next
  }
  print(i)
}
```

**Functions in R**

Functions are reusable blocks of code.

**Example**

```
add <- function(a, b){
  return(a + b)
}
result <- add(5, 3)
print(result)
```

**Data Frame in R**

A data frame is a table-like structure used to store data in rows and columns.

**Creating Data Frame**

```
student <- data.frame(
  Name = c("Amit", "Rahul", "Priya"),
  Age = c(20, 21, 19),
  Marks = c(85, 90, 88)
)
print(student)
```

**Accessing Data Frame**

student$Name

student$Marks

**Adding New Column**

student$Grade <- c("A", "A+", "A")

print(student)

**Removing Column**

student$Grade <- NULL

**Feature Engineering**

Feature Engineering is the process of transforming raw data into useful features for machine learning models.

It helps:

- Improve model accuracy
- Reduce complexity
- Handle categorical data
- Improve prediction quality

**Types of Feature Engineering**

| Technique | Purpose |
|---|---|
| Scaling | Normalize numerical data |
| Label Encoding | Convert text labels into numbers |
| One Hot Encoding | Convert categories into binary columns |
| Reduction | Reduce unnecessary features |

---

**Scaling**

Scaling standardizes the numerical values in a dataset.

**Min-Max Scaling**

Formula:

X_{scaled}=\frac{X-X_{min}}{X_{max}-X_{min}}

**Example**

```
x <- c(10,20,30,40,50)

scaled_x <- (x - min(x)) / (max(x) - min(x))

print(scaled_x)
```

**Standard Scaling**

Formula:

□genui□{"math_block_widget_always_prefetch_v2":{"content":"Z=\frac{X-\mu}{\sigma}"}}□

**Example**

```
x <- c(10,20,30,40,50)


scaled_x <- scale(x)

print(scaled_x)
```

**Label Encoding**

Label Encoding converts categorical text data into numeric values.

**Example**

```
colors <- c("Red","Blue","Green","Red")

encoded <- as.numeric(as.factor(colors))

print(encoded)
```

**One Hot Encoding**

One Hot Encoding creates separate binary columns for categories.

**Example**

```
data <- data.frame(Color = c("Red","Blue","Green"))

model.matrix(~Color-1, data)
```

**Output**

| ColorBlue | ColorGreen | ColorRed |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 0 |

**Feature Reduction**

Feature Reduction removes unnecessary features from the dataset.

**Advantages**

- Faster training
- Less memory usage
- Better performance
- Reduces overfitting

**Methods of Feature Reduction**

| Method | Description |
|---|---|
| Feature Selection | Select important features |

| Method | Description |
|---|---|
| PCA | Principal Component Analysis |
| Correlation Removal | Remove related features |

**Correlation Example**

```
cor(mtcars)
```

**Principal Component Analysis (PCA)**

PCA reduces dimensions of data while preserving important information.

**Example**

```
data <- iris[,1:4]
pca <- prcomp(data, scale=TRUE)
summary(pca)
```

**Built-in Datasets in R**

| Dataset | Purpose |
|---|---|
| iris | Flower classification |
| mtcars | Car analysis |
| airquality | Air pollution analysis |
| Titanic | Survival prediction |

**Importing Data in R**

**Read CSV File**

```
data <- read.csv("student.csv")
print(data)
```

**Exporting Data in R**

**Write CSV File**

```
write.csv(data, "output.csv")
```

**Basic Data Visualization in R**

**Bar Plot**

```
x <- c(10,20,30)

barplot(x)
```

**Pie Chart**

```
x <- c(20,30,40)


pie(x)
```

**Histogram**

```
hist(mtcars$mpg)
```

**Applications of R in Data Science**

- Machine Learning
- Predictive Analytics
- Statistical Analysis
- Data Visualization
- Artificial Intelligence
- Business Intelligence

**Advantages of R**

- Open Source
- Powerful statistical tools
- Large package support
- Excellent visualization libraries

**Limitations of R**

- High memory usage
- Slower for very large datasets
- Complex for beginners

**Conclusion**

R is one of the most powerful programming languages for Data Science and Machine Learning. Understanding data types, objects, control structures, data frames, and feature engineering techniques such as scaling, label encoding, one hot encoding, and reduction helps build accurate predictive models and efficient analytical systems.

**References**


| Sr. No. | Book Name | Author | Suitable For |
|---|---|---|---|
| 1 | *R for Data Science* | Hadley Wickham & Garrett Grolemund | Beginners & Data Science |
| 2 | *Hands-On Programming with R* | Garrett Grolemund | Beginners |
| 3 | *R in Action* | Robert Kabacoff | Intermediate Learners |
| 4 | *The Art of R Programming* | Norman Matloff | Advanced Concepts |
| 5 | *R for Everyone* | Jared P. Lander | Students & Academics |
| 6 | *Practical Data Science with R* | Nina Zumel & John Mount | Practical Applications |
| 7 | *An Introduction to R* | W.N. Venables & D.M. Smith | Official Beginner Guide |

Parul® University
Vadodara, Gujarat

NAAC GRADE A++


https://paruluniversity.ac.in/